

Improving the matrix multiplication exponent with modern optimization and AlphaEvolve

Emilien Dupont^{*,1}, Marvin Eisenberger^{*,1}, Borislav Kozlovskii^{*,1}, Abbas Mehrabian^{*,1}, Francisco J. R. Ruiz^{*,1}, Abigail See^{*,1}, Renfei Zhou^{*,2}, Josh Alman³, Virginia Vassilevska Williams⁴ and Matej Balog¹

^{*}Equal contribution in alphabetical order, ¹Google DeepMind, ²Carnegie Mellon University, ³Columbia University, ⁴MIT

The current best bounds on the matrix multiplication exponent ω are obtained through a refinement of the laser method called combination loss analysis (Duan et al., 2023; Williams et al., 2024; Alman et al., 2025). In this note, we address the optimization problem at the core of this approach and propose several improvements. First, we reformulate the optimization problem allowing us to solve it in a larger setting than was previously possible. Second, we leverage recent advances in machine learning to design a new optimization algorithm for this problem. Finally, we refine the resulting optimization algorithm with AlphaEvolve. Our combined approach yields an upper bound of $\omega < 2.371177$, improving the previous best bound of 2.371339.

1. Introduction

From accelerating machine learning computations to enabling realistic computer graphics, matrix multiplication is a fundamental operation underpinning critical applications in computer science. Despite its prominence, the computational complexity of matrix multiplication—the number of arithmetic operations needed to multiply large matrices—is unknown, and determining it is a major open question in theoretical computer science (Bläser, 2013).

The pioneering work of Strassen (Strassen, 1969) showed that two $n \times n$ matrices can be multiplied in sub-cubic time—specifically, $O(n^{\omega+o(1)})$ operations for $\omega < 2.81$ —spurring a line of work attempting to further reduce the complexity exponent ω (Pan, 1978; Bini et al., 1979; Schönhage, 1981; Romani, 1982; Coppersmith and Winograd, 1981; Strassen, 1986; Coppersmith and Winograd, 1990; Stothers, 2010; Williams, 2012; Le Gall, 2014; Alman and Williams, 2024; Duan et al., 2023; Williams et al., 2024; Alman et al., 2025). All improvements in the past 40 years rely on the *laser method*, a mathematical technique to indirectly design matrix multiplication algorithms. The current best bound is achieved by a refinement of the laser method called *combination loss analysis* (Duan et al., 2023), a technique that requires solving a non-convex optimization problem as part of the computer-assisted proof, and yields $\omega < 2.371339$ (Alman et al., 2025).

Here, we improve the bound to $\omega < 2.371177$ using a two-step approach. First, we leverage recent advances in machine learning and adjacent areas to address the non-convex optimization problem of combination loss analysis using a gradient descent approach; this alone improves the previous state-of-the-art (SOTA) bound by $\approx 0.97 \times 10^{-4}$. Second, we use AlphaEvolve (Novikov et al., 2025) to improve our optimization algorithm; this raises the improvement over the SOTA to $\approx 1.62 \times 10^{-4}$.

Duan et al. (2023)	2.371866
Williams et al. (2024)	2.371552
Alman et al. (2025)	2.371339
This note	2.371177

Table 1 | Recent improvements to ω .

The optimization problem at the core of combination loss analysis is formulated in [Alman et al. \(2025\)](#), where it was also shown that any feasible solution provides an upper bound on ω . To achieve our new bound, we target a slightly different optimization problem. Specifically, combination loss analysis has a parameter—the *maximum recursion level*, denoted by ℓ^* —that introduces a trade-off between the complexity of the optimization (which grows doubly exponentially in ℓ^*) and the best possible bound on ω it can achieve. The previous SOTA bound was found with $\ell^* = 3$ ([Alman et al., 2025](#)). In contrast, our gradient-based optimization, implemented in Jax ([Bradbury et al., 2018](#)), allows for hardware parallelization and can handle $\ell^* = 4$ (the number of optimizable parameters increases from approximately 25k to 7 million when moving from $\ell^* = 3$ to $\ell^* = 4$).

In this note, we describe the full optimization problem from [Alman et al. \(2025\)](#) in detail, explain how we numerically solved it and applied AlphaEvolve to it, and finally show how we rigorously certified the resulting omega bound.

2. Optimization problem

The high-level structure of the optimization problem can be captured by a rooted tree, where every node is associated with a collection of optimizable parameters. Integers $q \geq 1$ and $\ell^* \geq 2$ are fixed as hyperparameters. Intuitively, the tree structure describes a recursive way to decompose the tensor $CW_q^{\otimes 2^{\ell^*}}$ into smaller tensors, where CW_q is the Coppersmith-Winograd tensor ([Coppersmith and Winograd, 1990](#)); see [Alman et al. \(2025\)](#) for more details about this correspondence. We start by defining the tree structure, and will then define the optimizable parameters associated to its nodes.

2.1. Tree structure

Denote $[k] := \{1, \dots, k\}$. Each non-root node T in the tree is associated with a *level*, a *shape* s_T , and a *region* r_T , defined as follows.

- The *level* of a non-root node is a positive integer $\ell \in \{2, 3, \dots, \ell^*\}$, describing its depth in the tree; higher is closer to the root. Direct children of the root node have level ℓ^* , which is a hyperparameter fixed in advance.
- A level- ℓ *shape* is a triple $s = (s_X, s_Y, s_Z)$, where s_X, s_Y, s_Z are non-negative integers summing to 2^ℓ . We use

$$\mathcal{S}_\ell := \{(i, j, k) \in \mathbb{Z}_{\geq 0}^3 \mid i + j + k = 2^\ell\}$$

to denote the set of all level- ℓ shapes. We use *dimensions* X, Y, Z to refer to the indices of the three coordinates in a shape.

- There are six *regions*, indexed by an integer $r \in [6]$. Each region is associated with π_r , the r -th permutation over symbols $\{X, Y, Z\}$ in lexicographic order.

We call a non-root node T a *positive-shape node* if *all* coordinates in its shape s_T are positive. Otherwise it is a *zero-shape node*. The root node is denoted by G and does not have the aforementioned attributes—level, shape, and region.

Next, we define the tree structure by describing the child nodes for different types of nodes.

- **Root:** For every region $r \in [6]$ and level- ℓ^* shape $s \in \mathcal{S}_{\ell^*}$, the root G has a child at level ℓ^* with shape s and region r , denoted as $G[s, r]$.
- **Positive-shape node:** For every level- ℓ shape $s = (s_X, s_Y, s_Z)$, we define

$$\text{Split}(s) := \{u \in \mathcal{S}_{\ell-1} \mid 0 \leq u_X \leq s_X, 0 \leq u_Y \leq s_Y, 0 \leq u_Z \leq s_Z\}.$$

Fixing a positive-shape node T at level $\ell \geq 3$, for every region $r \in [6]$ and level- $(\ell - 1)$ shape $u \in \text{Split}(s_T)$, the node T has a child at level $\ell - 1$ with shape u and region r , denoted as $T[u, r]$. Note that the region index r of the child can be different from that of T itself.

- **Zero-shape nodes** and **level-2 positive-shape nodes** do not have child nodes. We call them the *leaves*.

2.2. Optimizable parameters

Next we list the free variables of the optimization problem, grouped by the associated node on the tree. For any finite set D , we use $\Delta(D)$ to denote the simplex of probability distributions on D :

$$\Delta(D) := \left\{ p : D \rightarrow [0, 1] \mid \sum_{x \in D} p(x) = 1 \right\}$$

Root node. The optimizable parameters associated with the root node G are:

- A distribution $A_G = (A_G^{(1)}, \dots, A_G^{(6)}) \in \Delta([6])$ over the six regions;
- For $r \in [6]$, a distribution $\alpha_G^{(r)} \in \Delta(\mathcal{S}_{\ell^*})$ over all level- ℓ^* shapes.

Positive-shape node. Each level- ℓ positive-shape node T with $\ell \geq 3$ is associated with the following parameters:

- A distribution $A_T = (A_T^{(1)}, \dots, A_T^{(6)}) \in \Delta([6])$ over the six regions;
- For $r \in [6]$, a distribution $\alpha_T^{(r)} \in \Delta(\text{Split}(s_T))$, where s_T is the shape of T .

Zero-shape node. For a level ℓ and an integer a with $0 \leq a \leq 2^\ell$, we define

$$C_{\ell, a} := \left\{ L \in \{0, 1, 2\}^{2^{\ell-1}} \mid \sum_{p=1}^{2^{\ell-1}} L_p = a \right\}.$$

This is the set of length- $2^{\ell-1}$ vectors over $\{0, 1, 2\}$ whose entries sum to a . A distribution over $C_{\ell, a}$ is called a *level- ℓ complete split distribution* of a .

For every level- ℓ zero-shape node T , let $W \in \{X, Y, Z\}$ be the first dimension where $s_{T,W}$ is nonzero. T is then associated with a complete split distribution $\beta_{T,W} \in \Delta(C_{\ell, s_{T,W}})$.

Level-2 node. The only remaining nodes are level-2 nodes with shapes $(1, 1, 2)$, $(1, 2, 1)$, or $(2, 1, 1)$, which are the only valid strictly positive shapes for level 2, since their coordinates must sum to $2^2 = 4$. Each such node is associated with a scalar $\mu_T \in [0, 1/2]$.

2.3. Derived quantities

With the free variables fixed, all remaining quantities can be computed deterministically. In the following, all logarithms and entropies are in base two. For a distribution ρ with finite support, we denote its entropy by

$$H(\rho) := - \sum_{x \in \text{supp}(\rho)} \rho(x) \log \rho(x).$$

We also use $H(p_1, p_2, \dots, p_k)$ to denote the entropy of a distribution over $[k]$ with probability masses $p_1, \dots, p_k$. If D is a set of shapes and $\rho \in \Delta(D)$, let ρ_W denote the marginal of ρ in coordinate W , namely, $\rho_W(w) := \sum_{a \in D: a_W=w} \rho(a)$. Define

$$H_D^{\max}(\rho) := \sup_{\substack{\rho' \in \Delta(D) \\ \rho'_W = \rho_W \text{ for } W \in \{X, Y, Z\}}} H(\rho'), \quad (1)$$

as well as the penalty notion

$$P_D(\rho) := H_D^{\max}(\rho) - H(\rho).$$

Masses. Each non-zero-shape node T of the tree has a real number $m_T \in [0, 1]$ associated with it, called its *mass*. The masses of non-root nodes are computed top-down as follows:

- The root G has $m_G = 1$.
- For the children of the root, we set

$$m_{G[s,r]} := A_G^{(r)} \cdot \alpha_G^{(r)}(s), \quad \forall s \in \mathcal{S}_{\ell^*}, r \in [6].$$

- For any positive-shape node T of level $\ell \geq 3$, we set

$$m_{T[u,r]} = m_T \cdot A_T^{(r)} \cdot \left(\alpha_T^{(r)}(u) + \alpha_T^{(r)}(s_T - u) \right), \quad \forall u \in \text{Split}(s_T), r \in [6].$$

Complete split distributions. Every level- ℓ non-root node T carries, for each dimension $W \in \{X, Y, Z\}$, a complete split distribution $\beta_{T,W} \in \Delta(\mathcal{C}_{\ell, s_{T,W}})$. These distributions are calculated as follows.

Let $\vec{0} := (0, 0, \dots, 0)$ and $\vec{2} := (2, 2, \dots, 2)$ denote vectors of length $2^{\ell-1}$. For a zero-shape node T , let $W_0 \in \{X, Y, Z\}$ be the first zero coordinate of s_T , W_1 be the first nonzero coordinate of s_T , and W_2 be the other coordinate. β_{T,W_0} is the point mass distribution at the length- $2^{\ell-1}$ vector $\vec{0}$; β_{T,W_1} was defined as optimizable parameters in section 2.2; $\beta_{T,W_2} := \beta_{T,W_1}^\vee$, where for any complete split distribution β , we define

$$\beta^\vee(\vec{2} - L) := \beta(L), \quad \forall L \in \text{supp}(\beta).$$

(Notice, in particular, that if the components of L sum to s_{T,W_1} , then the components of $\vec{2} - L$ sum to $2 \cdot 2^{\ell-1} - s_{T,W_1} = s_{T,W_2}$, keeping the mapping validly within the correct domain.)

For a positive-shape node T at level $\ell \geq 3$, we define

$$\beta_{T,W}^{(r)} := \sum_{u \in \text{Split}(s_T)} \alpha_T^{(r)}(u) \cdot (\beta_{T[u,r],W} \times \beta_{T[s_T-u,r],W}), \quad \forall r \in [6],$$

where $\times$ denotes the Cartesian product of complete split distributions; that is, if L_{left} and L_{right} are two sequences of length $2^{\ell-2}$ in supports of $\beta_{\text{left}} := \beta_{T[u,r],W}$ and $\beta_{\text{right}} := \beta_{T[s_T-u,r],W}$, respectively, then we form a new sequence L , of length $2^{\ell-1}$, by concatenating L_{left} and L_{right} , and letting

$$(\beta_{\text{left}} \times \beta_{\text{right}})(L) := \beta_{\text{left}}(L_{\text{left}}) \cdot \beta_{\text{right}}(L_{\text{right}}).$$

Then, the complete split distributions of T are calculated by

$$\beta_{T,W} := \sum_{r=1}^6 A_T^{(r)} \beta_{T,W}^{(r)}.$$

The only remaining case is a level-2 positive-shape node, which must have shape $(1, 1, 2)$, $(2, 1, 1)$, or $(1, 2, 1)$. For a node T with shape $(1, 1, 2)$, recall that $\mu_T \in [0, 1/2]$ is the only optimizable parameter associated with T . We let $\delta_{a,b}$ denote the point mass at $(a, b) \in \{0, 1, 2\}^2$ and set

$$\beta_{T,X} = \beta_{T,Y} = \frac{1}{2}\delta_{0,1} + \frac{1}{2}\delta_{1,0}, \quad \beta_{T,Z} = \mu_T\delta_{0,2} + \mu_T\delta_{2,0} + (1 - 2\mu_T)\delta_{1,1}. \quad (2)$$

Similarly, for a node T with shape $(2, 1, 1)$, we set

$$\beta_{T,Y} = \beta_{T,Z} = \frac{1}{2}\delta_{0,1} + \frac{1}{2}\delta_{1,0}, \quad \beta_{T,X} = \mu_T\delta_{0,2} + \mu_T\delta_{2,0} + (1 - 2\mu_T)\delta_{1,1},$$

and for a node T with shape $(1, 2, 1)$, we set

$$\beta_{T,X} = \beta_{T,Z} = \frac{1}{2}\delta_{0,1} + \frac{1}{2}\delta_{1,0}, \quad \beta_{T,Y} = \mu_T\delta_{0,2} + \mu_T\delta_{2,0} + (1 - 2\mu_T)\delta_{1,1}.$$

Retained exponent at root node. Next, we define how to calculate the *retained exponent* E_G associated with the root G , which will be obtained from intermediate quantities $E_G^{(r)}$ for regions $r \in [6]$. We start by introducing how to compute $E_G^{(1)}$ for region $r = 1$, where π_1 is the identity permutation. In all summations below, s ranges over $\mathcal{S}_{\ell^*}$. We will use the symbols $*$, $+$, and $<$ in our notation below as mnemonic placeholders corresponding to the dimensions, indicating the meanings “unconstrained / any value”, “strictly positive”, and “strictly less than the parent’s coordinate”, respectively. We define

$$\eta_{G,Y}^{(1)} := \sum_{s: s_Z=0} \alpha_G^{(1)}(s) \cdot H(\beta_{G[s,1],Y}) + \sum_{j=0}^{2^{\ell^*}} \alpha_G^{(1)}(*, j, +) \cdot H(\tilde{\beta}_{G,Y,* ,j,+}^{(1)}), \quad (3)$$

where

$$\alpha_G^{(1)}(*, j, +) := \sum_{s: s_Y=j, s_Z>0} \alpha_G^{(1)}(s), \quad \tilde{\beta}_{G,Y,* ,j,+}^{(1)} := \frac{1}{\alpha_G^{(1)}(*, j, +)} \sum_{s: s_Y=j, s_Z>0} \alpha_G^{(1)}(s) \cdot \beta_{G[s,1],Y}.$$

We then define

$$\eta_{G,Z}^{(1)} := \sum_{s: s_X=0 \text{ or } s_Y=0} \alpha_G^{(1)}(s) \cdot H(\beta_{G[s,1],Z}) + \sum_{k=0}^{2^{\ell^*}} \alpha_G^{(1)}(+, +, k) \cdot H(\tilde{\beta}_{G,Z,+,+,k}^{(1)}), \quad (4)$$

where

$$\alpha_G^{(1)}(+, +, k) := \sum_{s: s_X>0, s_Y>0, s_Z=k} \alpha_G^{(1)}(s), \quad \tilde{\beta}_{G,Z,+,+,k}^{(1)} := \frac{1}{\alpha_G^{(1)}(+, +, k)} \sum_{s: s_X>0, s_Y>0, s_Z=k} \alpha_G^{(1)}(s) \cdot \beta_{G[s,1],Z}.$$

To avoid the issue of denominators $\alpha_G^{(1)}(\cdot)$ being zero, we regard $0 \cdot \text{undefined} := 0$, so that Equation (3) and Equation (4) are always well-defined.

The quantity $E_G^{(1)}$ is then given by

$$E_G^{(1)} := \min \left\{ H((\alpha_G^{(1)})_X) - P_{\mathcal{S}_{\ell^*}}(\alpha_G^{(1)}), H(\tilde{\beta}_{G,Y,* ,*,*}^{(1)}) - \eta_{G,Y}^{(1)}, H(\tilde{\beta}_{G,Z,* ,*,*}^{(1)}) - \eta_{G,Z}^{(1)} \right\}, \quad (5)$$

where

$$\tilde{\beta}_{G,W,* ,*,*}^{(1)} := \sum_{s \in \mathcal{S}_{\ell^*}} \alpha_G^{(1)}(s) \cdot \beta_{G[s,1],W}, \quad \forall W \in \{X, Y, Z\}.$$

For $r = 2, \dots, 6$, we define $E_G^{(r)}$ by applying the same formula Equation (5) after relabeling the coordinates X, Y, Z as $\pi_r(X), \pi_r(Y), \pi_r(Z)$, and replacing the region index 1 with r . Finally, we set

$$E_G := \sum_{r=1}^6 A_G^{(r)} \cdot E_G^{(r)}.$$

Retained exponents at level $\ell \geq 3$. For each positive-shape node T at level $\ell \geq 3$, we will calculate intermediate quantities $E_{T,W}^{(r)}$ for each region $r \in [6]$ and $W \in \{X, Y, Z\}$. These quantities will later be aggregated to form the *retained exponent* of level ℓ . Note that the region index r in the calculation can be different from the region index r_T of node T itself. As in the above, we start by defining the quantities for region $r = 1$ where π_1 is the identity permutation. In the summations below, u ranges over $\text{Split}(s_T)$.

We first define

$$\begin{aligned} \eta_{T,Y}^{(1)} &:= \sum_{u: u_Z=0} \left(\alpha_T^{(1)}(u) + \alpha_T^{(1)}(s_T - u) \right) \cdot H\left(\beta_{T[u,1],Y}\right) \\ &\quad + \sum_{j=0}^{\min\{s_{T,Y}, 2^{\ell-1}\}} \left(\alpha_T^{(1)}(*, j, +) + \alpha_T^{(1)}(*, s_{T,Y} - j, <) \right) \cdot H\left(\bar{\beta}_{T,Y,* ,j,+}^{(1)}\right), \end{aligned} \quad (6)$$

where

$$\begin{aligned} \alpha_T^{(1)}(*, j, +) &:= \sum_{u: u_Y=j, u_Z>0} \alpha_T^{(1)}(u), \quad \alpha_T^{(1)}(*, s_{T,Y} - j, <) := \sum_{\substack{u: u_Y=s_{T,Y}-j, \\ u_Z < s_{T,Z}}} \alpha_T^{(1)}(u), \\ \bar{\beta}_{T,Y,* ,j,+}^{(1)} &:= \frac{1}{\alpha_T^{(1)}(*, j, +) + \alpha_T^{(1)}(*, s_{T,Y} - j, <)} \cdot \sum_{u: u_Y=j, u_Z>0} \left(\alpha_T^{(1)}(u) + \alpha_T^{(1)}(s_T - u) \right) \cdot \beta_{T[u,1],Y}. \end{aligned}$$

We also define

$$\begin{aligned} \eta_{T,Z}^{(1)} &:= \sum_{u: u_X=0 \text{ or } u_Y=0} \left(\alpha_T^{(1)}(u) + \alpha_T^{(1)}(s_T - u) \right) \cdot H\left(\beta_{T[u,1],Z}\right) \\ &\quad + \sum_{k=0}^{\min\{s_{T,Z}, 2^{\ell-1}\}} \left(\alpha_T^{(1)}(+, +, k) + \alpha_T^{(1)}(<, <, s_{T,Z} - k) \right) \cdot H\left(\bar{\beta}_{T,Z,+,+,k}^{(1)}\right), \end{aligned} \quad (7)$$

where

$$\begin{aligned} \alpha_T^{(1)}(+, +, k) &:= \sum_{u: u_X>0, u_Y>0, u_Z=k} \alpha_T^{(1)}(u), \quad \alpha_T^{(1)}(<, <, s_{T,Z} - k) := \sum_{\substack{u: u_X < s_{T,X}, u_Y < s_{T,Y}, \\ u_Z = s_{T,Z} - k}} \alpha_T^{(1)}(u), \\ \bar{\beta}_{T,Z,+,+,k}^{(1)} &:= \frac{1}{\alpha_T^{(1)}(+, +, k) + \alpha_T^{(1)}(<, <, s_{T,Z} - k)} \cdot \sum_{u: u_X>0, u_Y>0, u_Z=k} \left(\alpha_T^{(1)}(u) + \alpha_T^{(1)}(s_T - u) \right) \cdot \beta_{T[u,1],Z}. \end{aligned}$$

Again, as we define $0 \cdot \text{undefined} := 0$, Equation (6) and Equation (7) are well-defined even when some denominators $\alpha_T^{(1)}$ are zeros.

Then, $E_{T,W}^{(1)}$ are given by

$$\begin{aligned} E_{T,X}^{(1)} &:= m_T A_T^{(1)} \cdot \left(H\left((\alpha_T^{(1)})_X\right) - P_{\text{Split}(s_T)}\left(\alpha_T^{(1)}\right) \right), \\ E_{T,Y}^{(1)} &:= m_T A_T^{(1)} \cdot \left(H\left(\beta_{T,Y}^{(1)}\right) - \eta_{T,Y}^{(1)} \right), \\ E_{T,Z}^{(1)} &:= m_T A_T^{(1)} \cdot \left(H\left(\beta_{T,Z}^{(1)}\right) - \eta_{T,Z}^{(1)} \right). \end{aligned} \quad (8)$$

For regions $r = 2, \dots, 6$, we define $E_{T,\pi_r(X)}^{(r)}$, $E_{T,\pi_r(Y)}^{(r)}$, and $E_{T,\pi_r(Z)}^{(r)}$ by applying Equation (6), Equation (7), and Equation (8) after relabeling coordinates X, Y, Z as $\pi_r(X), \pi_r(Y), \pi_r(Z)$ and replacing region index 1 by r .

To calculate the *retained exponent* for level $\ell \in [3, \ell^*]$, we let $\mathcal{T}_\ell^+$ denote the set of positive-shape nodes at level ℓ . Then, the retained exponent at level ℓ is given by

$$E_\ell := \sum_{r=1}^6 \min \left\{ \sum_{T \in \mathcal{T}_\ell^+} E_{T,X}^{(r)}, \sum_{T \in \mathcal{T}_\ell^+} E_{T,Y}^{(r)}, \sum_{T \in \mathcal{T}_\ell^+} E_{T,Z}^{(r)} \right\}.$$

Retained exponent at level 2. For a positive level-2 node T of shape $(1, 1, 2)$, we define

$$(E_{T,X}, E_{T,Y}, E_{T,Z}) := m_T \cdot (1, 1, H(\mu_T, \mu_T, 1 - 2\mu_T)). \quad (9)$$

Similarly, for a node T of shape $(2, 1, 1)$, we define

$$(E_{T,X}, E_{T,Y}, E_{T,Z}) := m_T \cdot (H(\mu_T, \mu_T, 1 - 2\mu_T), 1, 1),$$

and for a node T of shape $(1, 2, 1)$, we define

$$(E_{T,X}, E_{T,Y}, E_{T,Z}) := m_T \cdot (1, H(\mu_T, \mu_T, 1 - 2\mu_T), 1),$$

Then, letting $\mathcal{T}_2^+$ denote the set of positive-shape nodes at level 2, the *retained exponent* at level 2 is given by

$$E_2 := \min \left\{ \sum_{T \in \mathcal{T}_2^+} E_{T,X}, \sum_{T \in \mathcal{T}_2^+} E_{T,Y}, \sum_{T \in \mathcal{T}_2^+} E_{T,Z} \right\}.$$

Local matrix size for zero-shape nodes. For every zero-shape node T , we let $W_0 \in \{X, Y, Z\}$ be the first zero coordinate of s_T and W_1 be the first nonzero coordinate of s_T . Then, we define the *local matrix size* of T , written $(M_{T,X}, M_{T,Y}, M_{T,Z})$, as

$$M_{T,W_0} := m_T \cdot \left(H(\beta_{T,W_1}) + \sum_{L \in \text{supp}(\beta_{T,W_1})} \beta_{T,W_1}(L) \cdot |\{p \in [2^{\ell-1}] \mid L_p = 1\}| \cdot \log q \right);$$

the other two coordinates in the local matrix size are zeros.

Local matrix size for $(1, 1, 2)$ -nodes. For a node T with shape $(1, 1, 2)$, we define its local matrix size as

$$(M_{T,X}, M_{T,Y}, M_{T,Z}) := m_T \cdot ((1 - 2\mu_T) \log q, (1 - 2\mu_T) \log q, 2\mu_T \log q). \quad (10)$$

Similarly, for a node T with shape $(2, 1, 1)$, we define its local matrix size as

$$(M_{T,X}, M_{T,Y}, M_{T,Z}) := m_T \cdot (2\mu_T \log q, (1 - 2\mu_T) \log q, (1 - 2\mu_T) \log q),$$

and for a node T with shape $(1, 2, 1)$, we define its local matrix size as

$$(M_{T,X}, M_{T,Y}, M_{T,Z}) := m_T \cdot ((1 - 2\mu_T) \log q, 2\mu_T \log q, (1 - 2\mu_T) \log q).$$

2.4. Final assembly

Finally, we define the *total retained exponent* as

$$E_{\text{total}} := E_G + E_2 + \sum_{\ell=3}^{\ell^*} E_{\ell},$$

and define the *total matrix size* as

$$M_{\text{total}} := \min \left\{ \sum_{T \in \mathcal{L}} M_{T,X}, \sum_{T \in \mathcal{L}} M_{T,Y}, \sum_{T \in \mathcal{L}} M_{T,Z} \right\},$$

where $\mathcal{L}$ is the set of leaves. We can now write the full optimization problem.

$ \begin{aligned} &\text{minimize} && \Omega \\ &\text{subject to} && E_{\text{total}} + M_{\text{total}} \cdot \Omega \geq 2^{\ell^* - 1} \log(q + 2), \\ &&& \text{all free variables lie in the domains stated in section 2.2.} \end{aligned} $	(11)
--	------

Theorem 1 (Alman et al. (2025)). *Any feasible solution of Equation (11) implies $\omega \leq \Omega$, where ω is the asymptotic matrix multiplication exponent.*

2.5. Dealing with maximum entropies

Among the quantities introduced above, $H_D^{\max}(\rho)$ is the only type that cannot be computed analytically. However, it is easy to see that replacing each occurrence of $H_D^{\max}$ in the computation of E_{total} with its upper bound can only decrease E_{total} , so any solution that satisfies the constraints in Equation (11) after this replacement is feasible for Equation (11) itself, and still yields a valid upper bound $\omega \leq \Omega$. In this section, we describe how to derive an upper bound of $H_D^{\max}(\rho)$.

For a set D of shapes and a probability distribution $\rho \in \Delta(D)$, a *valid certificate* for $H_D^{\max}(\rho)$ consists of:

1. a distribution $y \in \Delta(D)$ satisfying $y(a) > 0$ for every $a \in D$ and $y_W = \rho_W$ for every $W \in \{X, Y, Z\}$;
2. a real number λ_0 and, for each $W \in \{X, Y, Z\}$ and each marginal value $w \in \{a_W \mid a \in D\}$, a real number $\lambda_W(w)$ (these values play the role of Lagrange multipliers for the maximization defining $H_D^{\max}(\rho)$); and
3. a real number $\varepsilon \geq 0$ satisfying, for every $a = (a_X, a_Y, a_Z) \in D$,

$$|\log y(a) - (\lambda_0 + \lambda_X(a_X) + \lambda_Y(a_Y) + \lambda_Z(a_Z))| \leq \varepsilon. \quad (12)$$

Lemma 1. *For any valid certificate, we have $H(y) \leq H_D^{\max}(\rho) \leq H(y) + 2\varepsilon$.*

Proof. The left inequality follows from the definition: y has the same marginals as ρ , so it is feasible for the maximization defining $H_D^{\max}(\rho)$ (see Equation (1)).

For the right inequality, fix any $\rho' \in \Delta(D)$ with $\rho'_W = \rho_W$ for all $W \in \{X, Y, Z\}$; to prove the right inequality we need only show $H(\rho') \leq H(y) + 2\varepsilon$.

The entropy function H is concave on $\Delta(D)$, and since y is strictly positive, H is differentiable at y with $\nabla H(y)_a = -\log y(a) - \log e$ (recall that all logarithms are in base 2). Therefore

$$H(\rho') \leq H(y) + \langle \nabla H(y), \rho' - y \rangle.$$

Write $g(a) := \lambda_0 + \lambda_X(a_X) + \lambda_Y(a_Y) + \lambda_Z(a_Z)$, and view g , $\log y$, and the constant $\log e$ as vectors indexed by $a \in D$. Then,

$$\begin{aligned} \langle g + \log e, \rho' - y \rangle &= (\lambda_0 + \log e) \cdot \sum_{a \in D} (\rho'(a) - y(a)) + \sum_{W \in \{X, Y, Z\}} \sum_{a \in D} \lambda_W(a_W) \cdot (\rho'(a) - y(a)) \\ &= (\lambda_0 + \log e) \cdot (1 - 1) + \sum_{W \in \{X, Y, Z\}} \sum_{w \in \{a_W | a \in D\}} \lambda_W(w) \cdot (\rho'_W(w) - y_W(w)) = 0, \end{aligned}$$

where the last step uses $\rho'_W = \rho_W = y_W$ for each $W \in \{X, Y, Z\}$. Hence,

$$\langle \nabla H(y), \rho' - y \rangle = \langle g - \log y, \rho' - y \rangle \leq \varepsilon \cdot \|\rho' - y\|_1 \leq 2\varepsilon,$$

where the first inequality uses Equation (12) and the second one uses the fact that the ℓ_1 -distance between two probability distributions is at most 2. Thus, $H(\rho') \leq H(y) + 2\varepsilon$, proving the lemma. $\square$

3. Solving the optimization problem numerically

We wish to numerically minimize Ω from Equation (11) while satisfying all the constraints, with $q = 5$ and $\ell^* = 4$. In [Alman et al. \(2025\)](#), this was done (for $\ell^* = 3$) via a sequential quadratic programming (SQP) algorithm using the software package SNOPT ([Gill et al., 2002](#)). In contrast, we take a gradient-based approach to tackle the non-convex minimization. We develop an algorithm applying several techniques from machine learning and adjacent areas, such as optimal transport.

3.1. Differentiable objective

To apply a gradient-based optimization algorithm, we need a differentiable objective function to be optimized. However, obtaining a differentiable objective from Equation (11) is challenging due to the problem constraints. Since most of the parameters to be optimized are distributions, this creates the constraints that the probabilities must be non-negative and must add up to one. To sidestep that issue and obtain unconstrained optimization, we parameterize distributions in terms of their logits, and obtain the distribution by applying the softmax operation on the free parameters. We initialize the algorithm with random (logit) parameters.

The other relevant set of constraints comes from the maximum entropy distributions. Given an input probability distribution, we must find the distribution with maximum entropy that shares the same marginals as the input. This is reminiscent of a problem that typically arises in optimal transport, where the Sinkhorn-Knopp algorithm ([Sinkhorn and Knopp, 1967](#)) is used to obtain a stable and differentiable objective ([Cuturi, 2013](#)). Thus, unlike [Alman et al. \(2025\)](#), we do not treat the maximum entropy distributions (and the corresponding Lagrange multipliers) as free parameters to be optimized together with the rest of distributions; rather, we obtain the maximum entropy distributions (and the Lagrange multipliers) using the Sinkhorn-Knopp algorithm.

We compute the gradients of the objective using automatic differentiation. To improve stability, we employ implicit differentiation for backpropagating through the Sinkhorn-Knopp algorithm ([Cuturi et al., 2020](#); [Eisenberger et al., 2022](#)). We update the parameters using Adam ([Kingma and Ba, 2015](#)).

3.2. Software implementation

To obtain a fast algorithm that can leverage hardware platforms (such as GPUs) and scale up to level $\ell^* = 4$ (with nearly 7 million parameters to optimize) we implement our algorithm in Jax ([Bradbury et al., 2018](#)), applying some techniques to improve efficiency. The key to an effective speed-up is

finding a new representation for the problem, switching from a loop over graph nodes (as done in the SQP implementation from [Alman et al. \(2025\)](#)) to a naturally parallelizable computation over tensors. In the implementation from [Alman et al. \(2025\)](#), the free parameters are stored in the nodes of a graph where updates happen through message-passing (both from child nodes to parent nodes and vice versa). The graph is non-uniform enough (e.g., different nodes have different number of children and different types of them) to make parallelization hard, but we solve this via two techniques. Firstly, we introduce phantom nodes in the graph with masking; as a trade-off, we pay the cost of increasing the number of optimization parameters by up to a factor of 3. Secondly, we cluster the nodes into a limited number of highly specialized groups (“stages”). With these techniques, we are able to represent the full graph with multi-dimensional tensors, enabling parallel processing over a number of axes (up to 10), challenging the limits of tensor-processing backends.

3.3. Applying AlphaEvolve

We use AlphaEvolve ([Novikov et al., 2025](#)) to further improve the optimization algorithm. Specifically, we let AlphaEvolve modify the optimization program, which is then executed (taking approximately 5 hours on a single GPU) to output a bound on ω . AlphaEvolve then evolves the code to minimize ω . We found improved results by using AlphaEvolve’s “evolving constructions” feature, where the optimization algorithm at each generation starts at the best solution point found by the parent algorithm.

4. Rigorous verification of the ω upper bound

To rigorously certify our ω bound, we run a separate verification step computing all quantities in rational arithmetic to guard against floating point errors. More specifically, we round the floating point solution obtained at the end of the optimization to rational numbers, ensuring the maximum-entropy certificates stay valid. We then evaluate all derived quantities in exact rational arithmetic, and replace each logarithm with a rational bound rounded in the proper direction that ensures each constraint in Equation (11) is satisfied, so the certified bounds are free from numerical errors. We are preparing a repository in which we will release the verification code and our discovered solution.

5. Discussion

In this note, we improve the SOTA upper bound on ω to 2.371177. Our improvement is comparable in magnitude to most improvements in the last 40 years since $\omega < 2.376$ was attained by [Coppersmith and Winograd \(1990\)](#). We achieve this by leveraging modern optimization techniques and AlphaEvolve to design a better optimization algorithm for the problem described by [Alman et al. \(2025\)](#). While further modest improvements may be obtained in this manner, achieving larger improvements to ω likely requires new mathematical ideas and is an exciting area of research.

Acknowledgments. The work presented in this note was motivated by initial conversations during the Complexity and Linear Algebra program at Simons Institute for the Theory of Computing in Fall 2025.

References

- J. Alman and V. V. Williams. A refined laser method and faster matrix multiplication. *TheoretCS*, 3 (11261), 2024.
- J. Alman, R. Duan, V. V. Williams, Y. Xu, Z. Xu, and R. Zhou. *More Asymmetry Yields Faster Matrix Multiplication*, pages 2005–2039. Society for Industrial and Applied Mathematics, 2025. doi: 10.1137/1.9781611978322.63. URL <https://epubs.siam.org/doi/abs/10.1137/1.9781611978322.63>.
- D. Bini, M. Capovani, F. Romani, and G. Lotti. $O(n^{2.7799})$ complexity for $n \times n$ approximate matrix multiplication. *Information Processing Letters*, 8(5):234–235, 1979. ISSN 0020-0190.
- M. Bläser. Fast matrix multiplication. *Theory of Computing*, pages 1–60, 2013.
- J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>.
- D. Coppersmith and S. Winograd. On the asymptotic complexity of matrix multiplication. In *22nd Annual Symposium on Foundations of Computer Science (sfcs 1981)*, pages 82–90, 1981.
- D. Coppersmith and S. Winograd. Matrix multiplication via arithmetic progressions. *Journal of Symbolic Computation*, 9(3):251–280, 1990. ISSN 0747-7171. Computational algebraic complexity editorial.
- M. Cuturi. Sinkhorn distances: lightspeed computation of optimal transport. In *International Conference on Neural Information Processing Systems*, pages 2292–2300, 2013.
- M. Cuturi, O. Teboul, J. Niles-Weed, and J.-P. Vert. Supervised quantile normalization for low-rank matrix approximation. In *International Conference on Machine Learning*, 2020.
- R. Duan, H. Wu, and R. Zhou. Faster matrix multiplication via asymmetric hashing. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 2129–2138, 2023. doi: 10.1109/FOCS57990.2023.00130.
- M. Eisenberger, A. Toker, L. Leal-Taixé, F. Bernard, and D. Cremers. A unified framework for implicit Sinkhorn differentiation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 499–508, 2022.
- P. E. Gill, W. Murray, and M. A. Saunders. SNOPT: An SQP algorithm for large-scale constrained optimization. *SIAM Journal on Optimization*, 12(4):979–1006, 2002.
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- F. Le Gall. Algebraic complexity theory and matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation, ISSAC ’14*, page 23, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450325011.
- A. Novikov, N. Vű, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv*, 2025. URL <https://arxiv.org/abs/2506.13131>.

- V. Y. Pan. Strassen's algorithm is not optimal trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations. In *19th Annual Symposium on Foundations of Computer Science*, pages 166–176, 1978.
- F. Romani. Some properties of disjoint sums of tensors related to matrix multiplication. *SIAM Journal on Computing*, 11(2):263–267, 1982.
- A. Schönhage. Partial and total matrix multiplication. *SIAM Journal on Computing*, 10(3):434–455, 1981.
- R. Sinkhorn and P. Knopp. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific Journal of Mathematics*, 21:343–348, 1967.
- A. J. Stothers. *On the complexity of matrix multiplication*. Phd thesis, University of Edinburgh, Edinburgh, United Kingdom, 2010.
- V. Strassen. Gaussian elimination is not optimal. *Numerische mathematik*, 13(4):354–356, 1969.
- V. Strassen. The asymptotic spectrum of tensors and the exponent of matrix multiplication. In *27th Annual Symposium on Foundations of Computer Science*, pages 49–54, 1986.
- V. V. Williams. Multiplying matrices faster than Coppersmith-Winograd. In *Proceedings of the Forty-Fourth Annual ACM Symposium on Theory of Computing*, STOC '12, page 887–898, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450312455.
- V. V. Williams, Y. Xu, Z. Xu, and R. Zhou. *New Bounds for Matrix Multiplication: from Alpha to Omega*, pages 3792–3835. Society for Industrial and Applied Mathematics, 2024. doi: 10.1137/1.9781611977912.134. URL <https://epubs.siam.org/doi/abs/10.1137/1.9781611977912.134>.